

UNIVERSIDAD EUROPEA DEL ATLÁNTICO
GRADO EN
Ingeniería Informática



INTELIGENCIA ARTIFICIAL

PROYECTO FINAL OCR

Trabajo realizado por:
Paula Qing Diestro Gutiérrez

Profesor:
Jose Manuel Breñosa

SANTANDER 09/01/2026

1. Introducción

El Reconocimiento Óptico de Caracteres (OCR, *Optical Character Recognition*) es una rama clásica de la Inteligencia Artificial cuyo objetivo es convertir texto presente en imágenes en información digital editable. Este tipo de sistemas tiene aplicaciones directas en digitalización de documentos, lectura automática de formularios, reconocimiento de matrículas, análisis de manuscritos y accesibilidad, entre otros.

El reto planteado en esta asignatura consiste en diseñar e implementar un sistema OCR completo desde cero, sin utilizar librerías externas que ya resuelvan el problema de reconocimiento de texto (como Tesseract), pero permitiendo el uso de librerías de propósito general para procesamiento de imágenes y aprendizaje automático. Además, el sistema debe ser capaz de reconocer tanto caracteres digitales como caracteres escritos a mano, lo cual añade una complejidad significativa debido a la variabilidad inherente a la escritura manual.

Para afrontar este problema se ha optado por un enfoque híbrido que combina:

- Técnicas clásicas de procesamiento digital de imágenes (binarización, filtrado, segmentación).
- Un modelo de *Deep Learning*, concretamente una Red Neuronal Convolutiva (CNN), entrenada para clasificar caracteres individuales.

El sistema desarrollado sigue una arquitectura modular, donde cada etapa del OCR está claramente separada:

1. Carga y preparación del dataset.
2. Preprocesamiento de imágenes de entrada.
3. Segmentación de líneas y caracteres.
4. Reconocimiento individual de caracteres mediante una CNN.
5. Reconstrucción del texto final y evaluación de la confianza.

El dataset utilizado está compuesto por imágenes de letras mayúsculas, minúsculas y números proporcionados por el conjunto de alumnos a través de una carpeta compartida en Google Drive. Este enfoque permite entrenar un modelo adaptado específicamente al problema planteado, sin recurrir a soluciones externas ya entrenadas.

El desarrollo y ejecución del proyecto se ha realizado íntegramente en Google Colab, facilitando el acceso directo a los datos almacenados en Google Drive.

2. Solución aportada

2.1 Arquitectura general del sistema

El sistema OCR implementado sigue el flujo clásico de un OCR basado en caracteres:

Imagen de entrada

↓

Preprocesamiento

↓

Segmentación de líneas

↓

Segmentación de caracteres

↓

Normalización de cada carácter

↓

Clasificación con CNN

↓

Reconstrucción del texto

Cada uno de estos pasos ha sido implementado de forma independiente para facilitar el mantenimiento, la experimentación y futuras mejoras.

2.2 Carga y preparación del dataset

El dataset se organiza en carpetas por categorías:

- mayusculas
- minusculas
- numeros

Cada carpeta contiene subcarpetas con el nombre del carácter correspondiente (por ejemplo, A, b, 7), y dentro de ellas múltiples imágenes del mismo carácter.

La clase DatasetLoader se encarga de:

- Recorrer la estructura de carpetas.

- Asignar una etiqueta numérica única a cada carácter.
- Convertir las imágenes a escala de grises.
- Redimensionar las imágenes a un tamaño fijo de 32×32 píxeles, manteniendo la proporción mediante *padding*.
- Normalizar los valores de píxel al rango [0, 1].

Este proceso garantiza que todas las imágenes de entrada al modelo tengan el mismo formato, condición indispensable para entrenar una red neuronal.

2.3 Data Augmentation

Dado que la escritura manual presenta gran variabilidad, se ha aplicado data augmentation ligero para mejorar la capacidad de generalización del modelo. Concretamente, para cada imagen original se generan dos versiones adicionales mediante:

- Rotación de +3°
- Rotación de -3°

De este modo, el tamaño del dataset se triplica, simulando variaciones realistas en la inclinación de los caracteres sin introducir distorsiones excesivas.

2.4 Modelo de reconocimiento: Red Neuronal Convolutiva

Para el reconocimiento de caracteres se ha diseñado una CNN profunda, optimizada para obtener un buen equilibrio entre precisión y tiempo de entrenamiento.

Estructura del modelo:

- Tres bloques convolucionales con:
 - Capas Conv2D
 - Normalización (BatchNormalization)
 - Submuestreo (MaxPooling)
 - Regularización mediante Dropout
- Capas densas finales para la clasificación.
- Capa de salida con función softmax, con tantas neuronas como caracteres distintos.

La función de pérdida utilizada es `sparse_categorical_crossentropy`, adecuada para clasificación multiclase, y el optimizador seleccionado es Adam, con una tasa de aprendizaje ajustada dinámicamente mediante *ReduceLROnPlateau*.

Para evitar sobreajuste se utilizan:

- *Early Stopping*, monitorizando la precisión en validación.
- *Dropout* en distintas capas del modelo.

2.5 Preprocesamiento de imágenes de entrada

Antes de segmentar y reconocer caracteres, la imagen de entrada pasa por un pipeline de preprocesamiento implementado en la clase `ImagePreprocessor`, que incluye:

1. Conversión a escala de grises.
2. Eliminación de ruido mediante filtro bilateral.
3. Binarización automática utilizando el método de Otsu.
4. Corrección de inclinación (deskew) mediante el cálculo del ángulo mínimo que encierra el texto.

Este proceso mejora significativamente la calidad de la segmentación posterior.

2.6 Segmentación de líneas y caracteres

La segmentación se realiza en dos fases:

- **Segmentación de líneas:** Se calcula la proyección horizontal de la imagen binarizada y se detectan regiones con suficiente densidad de píxeles.
- **Segmentación de caracteres:** Para cada línea se calcula la proyección vertical, identificando los límites de cada carácter individual.

Cada carácter segmentado se limpia eliminando bordes innecesarios y se redimensiona nuevamente a 32×32 píxeles con *padding*, asegurando coherencia con los datos de entrenamiento.

2.7 Reconstrucción del texto

Cada carácter segmentado se clasifica de forma independiente usando la CNN entrenada.

El sistema devuelve:

- El texto reconocido.
- Una confianza promedio basada en las probabilidades de salida del modelo.

Cuando la confianza de un carácter es inferior a un umbral definido, se sustituye por el símbolo ?, indicando incertidumbre.

3. Registro de resultados

- **03-01-26 18:30** Prueba inicial con:

- Script para cargar y organizar tu dataset de Drive
- CNN para reconocimiento de caracteres
- Sistema de preprocesamiento y segmentación

Resultado: accuracy = 55.65%. Anotación: modelo insuficiente para reconocer escritura manual.

- **04-01-26 17:15** Intento de mejora de la precisión del código mejorando el modelo. Los cambios en los epochs para aumentar el tiempo de aprendizaje y ajuste del batch_size=12 para aprender de forma más precisa.

Resultado: accuracy sigue siendo muy bajo. Anotación: sigue sin haber una clara mejora.

- **07-01-2026 17:00** Incorporación de dos modelos, uno para el texto digital y otro para el texto escrito.

Anotación: Código demasiado complejo. Tardaba demasiado tiempo entrenando el modelo.

- **07-01-2026 20:00** Vuelta al modelo inicial optimizando el código.

Resultado: accuracy < 70%. Anotación: modelo estable, pero sin un buen reconocimiento de los caracteres tanto digitales como manuscritos.

- **08-01-2026 18:30** Cambios en el código:

- Celda 3: Añadir 3 métodos a la clase OCRSystem (1 modificado + 2 nuevos)
- Celda 4: Reemplazar completamente la función train_model()

Resultado: accuracy > 85%. Anotación: La precisión es alta, pero sigue sin ser capaz de reconocer correctamente los caracteres.

4. Conclusiones

En este proyecto se ha desarrollado un sistema OCR completo que no es del todo capaz de reconocer texto digital y manuscrito a partir de imágenes, cumpliendo todas las restricciones impuestas en el enunciado.

Se ha demostrado que, combinando técnicas clásicas de procesamiento de imágenes con modelos modernos de Deep Learning, es complicado construir soluciones robustas.

Alguna observación respecto al reto es la gran variación de formato de los datasets aportados por los alumnos. Cada uno subido en un formato y tipografía muy distintos, llegando alguno a ser difícil de reconocer a simple vista por las propias personas.

Como posibles mejoras futuras se plantean:

- Uso de modelos secuenciales (CNN + LSTM) para reconocer palabras completas.
- Inclusión de un corrector ortográfico posterior.
- Entrenamiento con un dataset más amplio y diverso.
- Mejora de la segmentación para textos con caracteres conectados.

En conclusión, el sistema desarrollado no constituye una solución funcional, extensible y bien fundamentada desde el punto de vista de la Inteligencia Artificial, lo cual no se ha podido cumplir con los objetivos del reto propuesto.